Chiron

Repository Simplification and Strategic Runtime Reframing Before Migration Execution

Progress Report 7 • 22 Dec 2025 — 4 Jan 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 7

Title: Repository Simplification and Strategic Runtime Reframing Before Migration Execution
Reporting Window: 22 Dec 2025 — 4 Jan 2026
Submission Context: Master’s Research Progress Update

Abstract

Progress Report 7 was a low-throughput but strategically decisive period. At the implementation surface, the period was dominated by repository cleanup and structural simplification rather than broad feature expansion. At the architecture-decision surface, this same window surfaced a clearer concern: the active orchestration path was carrying more framework weight than the project needed for its actual execution goals.

The period is therefore best interpreted as an inflection point. It did not execute the full runtime transition, but it established the practical and conceptual conditions that made the next transition phase both necessary and actionable.

---

1. Period Character: Controlled Scope, High Leverage

This reporting window combined compact development activity with high strategic impact. Delivery focus shifted away from visible capability growth and toward three tightly scoped outcomes:

- reduce structural overhead,
- sharpen migration readiness,
- and clarify which orchestration responsibilities should remain first-class inside Chiron.

This mode was deliberate. It preserved implementation stability while creating decision clarity for the next phase.

---

2. Repository Simplification as Pre-Migration Work

The largest technical movement in the period was repository reshaping through external subtree burden reduction and related cleanup. The practical effect was a leaner baseline with fewer inherited maintenance surfaces and less coordination noise.

This work is significant because it changed migration economics. By reducing structural drag first, subsequent runtime decisions could be evaluated against Chiron's own constraints rather than against accumulated repository complexity.

---

3. Emergent Runtime Mismatch: Mastra Scope vs Chiron Needs

During this period, implementation and planning discussions converged on a recurring mismatch: the active Mastra-centered path appeared increasingly heavy relative to the project's required primitives. The issue was not framework quality in isolation; it was fit-for-purpose alignment within Chiron's workflow-engine trajectory.

The practical insight that emerged was that Chiron required tighter ownership over execution semantics, approval behavior, and scoping boundaries than a generalized orchestration layer was conveniently providing in this context.

This report should therefore document the period as the point where that mismatch became explicit and decision-relevant.

---

4. Early Directional Shift Toward Leaner Runtime Primitives

A full migration was not completed in this window. However, the direction of travel became clearer: a leaner, more controlled runtime approach was emerging as the better fit, with Effect and AI-SDK beginning to stand out as promising directions for service boundaries and LLM interaction [2], [3].

Historically, this matters because later migration work should be read not as a sudden reaction, but as follow-through on a direction that became clearer in this period.

---

5. Scoping Pressure Becomes the Core Architectural Question

A second major signal in this period was a persistent scoping problem that had been building across the previous cycle: how to bound and reason about work at project, workflow, and step levels without ambiguity.

In this report window, that pressure intensified and became central to architecture thinking. The final formal answer did not fully materialize yet, but the problem definition became sharper:

- execution context needed explicit boundaries,

- transition behavior needed stronger control semantics,
- and work-state progression needed clearer authority ownership.

This period is better understood as the pressure point that later fed into layered lifecycle/state formalization.

---

6. Optimizer Direction Narrows Before Migration

This period also matters because the optimizer question became more disciplined. Earlier DSPy-style exploration had surfaced several optimizer families, but by this stage the project was no longer treating them as equally ready for Chiron's next implementation phase.

The most plausible near-term optimizer became **MiPRO**. Because it is built around improving instructions and demonstrations for structured tasks with explicit evaluation metrics, it remained the most realistic candidate for the AX-oriented layer Chiron was considering adding later [8], [9]. In practical terms, that made it a better fit for things like structured classification, constrained selection, and workflow-path recommendation than more open-ended optimizer strategies.

GEPA remained intellectually attractive, especially because reflective evolution and Pareto-style candidate selection suggest strong potential for multi-objective outputs [10], [11]. However, that same richness also made it more complicated, and in this period it should be understood as a deferred direction rather than the optimizer the project was preparing to operationalize next.

ACE was even more future-facing. Its generator-reflector-curator playbook model matched the idea of agents gradually learning preferred rules and reusable practices over repeated workflows and multiple projects [12], [13]. But that same agent-loop orientation meant ACE belonged to a later stage of Chiron's evolution, once the runtime and methodology boundaries were stronger and long-lived agent memory could be introduced coherently.

Historically, this is the important distinction: by the end of this window, the optimizer conversation had stopped being "which advanced technique sounds interesting?" and had become "which optimizer is realistic for the next execution phase?" In that narrower frame, MiPRO remained viable, while GEPA and ACE were still deferred.

---

7. Product-Surface Changes During This Window

User-facing behavior changes were intentionally narrow. The period did not attempt broad workflow-surface expansion. Instead, it prioritized structural readiness and architecture clarity under controlled delivery scope.

This distinction is important for chronology: Progress Report 7 should be read as preparation and reframing, not as migration completion.

---

8. Stack Status at End of Progress Report 7

At the end of this period, the active stack in implementation practice remained:

- **Desktop host:** Tauri [5]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + tRPC [6], [7]
- **AI orchestration/runtime path:** Mastra + AX (under active reassessment) [1], [4]
- **Data layer:** Drizzle ORM with PostgreSQL
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo

The key update was directional rather than completed replacement: runtime reassessment became explicit and began shaping the next execution window.

---

9. Why This Period Matters

Progress Report 7 matters because it marks the transition from implicit discomfort to explicit architecture reframing. The period reduced repository complexity and, in parallel, clarified two strategic realities:

1. orchestration fit had to be re-evaluated against Chiron’s concrete needs,
2. scoping and execution-boundary control had become first-order design constraints.

This made the next report’s migration activity interpretable as planned follow-through rather than abrupt redirection.

---

10. Open Questions Entering Progress Report 8

- Which runtime responsibilities should be owned directly by Chiron versus delegated to external orchestration abstractions?
- How should migration sequencing preserve workflow reliability while replacing core execution dependencies?

- What boundary model should govern project-, workflow-, and step-level scope to prevent context drift?
- How should approval, transition, and execution semantics be represented so later lifecycle/state formalization remains consistent?

These questions define the handoff from preparation/reframing to migration execution.

Conclusion

Progress Report 7 is best understood as a strategic inflection period. It delivered structural simplification work, exposed an orchestration fit mismatch, and elevated scoping control from background concern to central architecture driver.

The period did not finalize the later layered lifecycle/state solution, but it established the conditions under which that solution became necessary. In the historical sequence, this report is the bridge between cleanup and the migration work that follows.

References

[1] Mastra, “Mastra Documentation,” 2025. [Online]. Available: https://mastra.ai/docs
[2] Effect Authors, “Effect Documentation,” 2025. [Online]. Available: https://effect.website/
[3] Vercel, “AI SDK Documentation,” 2025. [Online]. Available: https://ai-sdk.dev/docs
[4] axllm, “ax,” GitHub repository (`@ax-llm/ax`), 2025. [Online]. Available: https://github.com/axllm/ax
[5] Tauri Programme, “Tauri,” Documentation, 2025. [Online]. Available: https://tauri.app/
[6] Hono Authors, “Hono,” Documentation, 2025. [Online]. Available: https://hono.dev/
[7] tRPC Authors, “tRPC,” Documentation, 2025. [Online]. Available: https://trpc.io/docs
[8] DSPy, “MIPROv2,” Documentation, 2025. [Online]. Available: https://dspy.ai/api/optimizers/MIPROv2/
[9] S. Opsahl-Ong et al., “Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs,” 2024. [Online]. Available: https://arxiv.org/abs/2406.11695
[10] DSPy, “GEPA Overview,” Documentation, 2025. [Online]. Available: https://dspy.ai/api/optimizers/GEPA/overview/
[11] L. A. Agrawal et al., “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning,” 2025. [Online]. Available: https://arxiv.org/abs/2507.19457
[12] Q. Zhang et al., “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” 2025. [Online]. Available: https://arxiv.org/abs/2510.04618

[13] Ax LLM, “Agentic Context Engineering (ACE),” Documentation, 2025. [Online]. Available: https://axllm.dev/ace/